



FUELFLOW FSMS
RELEASE READINESS ASSESSMENT

Senior QA Report

End-to-end release readiness assessment

Assessment date

15 August 2026

Build assessed

77cc343 - agent/publish-fuelflow-project

RELEASE VERDICT: NOT READY

One high-severity release blocker requires correction before deployment.

1. Executive summary

FuelFlow's implemented functional slice is broadly healthy. The local six-service stack runs, both tenant and vendor authentication work, all visible tenant/vendor workspaces render against the live PostgreSQL API, the main operational backend workflow passes, responsive navigation works, and both production images build successfully.

The release gate fails because the documented backend test command is not isolated from live services. Although `phpunit.xml` declares an in-memory database, synchronous queue, and array mailer, the effective test runtime used SQLite together with the live Redis queue and configured SMTP mailer. This QA run caused 34 onboarding notification jobs to enter the live queue and fail after retry because their serialized models referenced the test SQLite connection. In another failure mode, the same configuration leak could send test notifications through a real mail account.

A second functional defect causes protected API routes to return HTTP 500 instead of 401 when an unauthenticated client does not send `Accept: application/json`. Two lower-risk quality gaps were also recorded.

Finding summary

Severity	Count	Release impact
High	1	Release blocker
Medium	2	Fix before external API use; automate before sustained delivery
Low	1	Schedule with next frontend quality pass

2. Scope and environment

The assessment covered:

- React 19/TypeScript tenant and vendor interfaces
- Laravel 13 API, authentication, authorization, validation, and operational workflows
- PostgreSQL 17 data layer and all repository migrations
- Redis cache/queue connectivity
- Queue worker and scheduler behavior
- Notification/report scheduling logic through automated tests and runtime inspection
- Local and production Docker Compose configuration
- Production API and web image builds
- Dependency audits, source formatting, lint, type checking, unit/feature tests
- Desktop and 390 px mobile browser smoke tests
- Basic API security contracts: authentication, CSRF, CORS, and secret scanning

Runtime versions observed included Docker 29.6.2 and PHP 8.4.24. The local stack consisted of PostgreSQL, Redis, API, queue worker, scheduler, and Vite web services.

3. Release gate results

Area	Result	Evidence
Container runtime	Pass	All six local services running; PostgreSQL, Redis, and API reported healthy
Database migrations	Pass	All 17 migrations reported Ran
API health	Pass	GET /api/v1/health returned 200 with the expected service/currency payload
Backend automated tests	Pass with isolation defect	39 tests, 336 assertions passed; see QA-001
Backend format/manifest	Pass	Pint passed 149 files; Composer manifest valid
Frontend tests	Pass with coverage gap	3 files, 8 tests passed; see QA-004
Frontend lint	Pass with warnings	0 errors, 2 hook-dependency warnings; see QA-003
TypeScript/production bundle	Pass	tsc -b and Vite build completed
Dependency security	Pass	Composer and production npm audits reported no known advisories
Production packaging	Pass	Production Compose parsed; API and web Docker images built successfully
Tenant browser flow	Pass	Tenant login plus 16 tenant routes rendered without alerts, overlays, or browser errors
Vendor browser flow	Pass	Vendor login plus dashboard, company, password, and onboarding-validation views worked
Responsive smoke	Pass	All 16 tenant routes rendered at 390 px without document-level horizontal overflow; mobile navigation opened, navigated, and closed
Background scheduler	Pass	Report dispatch ran each minute; license-alert schedule was registered
Queue/runtime isolation	Fail	34 failed notification jobs created by the test run; see QA-001
Unauthenticated API contract	Fail	Protected routes return 500 without a JSON Accept header; see QA-002

4. Detailed findings

QA-001 - Tests use the live Redis queue and SMTP configuration

Severity: High **Priority:** P0 / release blocker **Status:** Reproducible

Evidence

1. The documented command completed green:

```
docker compose exec -T api php artisan test --compact
Tests: 39 passed (336 assertions)
```

2. A temporary diagnostic test executed inside the same PHPUnit runtime reported:

```
{"database":"sqlite","queue":"redis","mail":"smtp","cache":"array","session":"array"}
```

Expected values from `phpunit.xml` were `queue=sync` and `mail=array`.

3. The queue worker processed and failed jobs immediately after the test run. The live database contained **34 failed jobs**:

- 24 App\Notifications\UserOnboardedNotification
- 10 App\Notifications\CompanyOnboardedNotification

4. The common failure was:

```
SQLiteDatabaseDoesNotExistException: Database file at path [fuelflow] does not exist.
```

This is consistent with models serialized by the SQLite test runtime being restored by the live PostgreSQL/Redis worker.

5. `phpunit.xml` declares the safe mail/queue drivers, but `tests/bootstrap.php` explicitly forces only database, cache, and session settings. `VendorPlatformApiTest.php` performs onboarding repeatedly without globally faking notifications.

Impact

- Running the repository's own test command pollutes the live Redis queue and `failed_jobs` table.
- Test notifications may reach a real SMTP provider if a queued payload does not fail earlier.
- A green test result hides failed asynchronous work and gives false release confidence.
- Local/test data can leak into long-running workers through serialized model references.

Recommended fix

1. Force `QUEUE_CONNECTION=sync` and `MAIL_MAILER=array` in `tests/bootstrap.php` before the Laravel application boots, using the same `putenv`, `$_ENV`, and `$_SERVER` pattern as the database overrides.
2. Add a permanent safety test asserting the effective database, queue, mail, cache, and session drivers.
3. Add `Notification::fake()` or `Queue::fake()` to vendor tests that are not explicitly testing delivery.
4. Run the suite while monitoring Redis and assert that its queue length does not change.
5. After the fix is verified, review and then remove/retry the 34 failed QA jobs according to the team's data-retention policy.

Acceptance criteria

- The diagnostic configuration is `sqlite/sync/array/array/array`.
- A full test run creates zero Redis jobs and zero new `failed_jobs` records.
- No SMTP connection is attempted during automated tests.

QA-002 - Unauthenticated API requests can return 500 instead of 401

Severity: Medium **Priority:** P1 **Status:** Reproducible

Reproduction

```
GET /api/v1/me
Accept header omitted
Actual: HTTP 500, "Route [login] not defined."
Expected: HTTP 401 with an API-safe error response
```

The behavior also reproduced on `/api/v1/dashboard`, `/api/v1/vendor/me`, `/api/v1/users`, and `/api/v1/audit-events`. Adding `Accept: application/json` returned the correct 401 response.

`bootstrap/app.php` requests JSON rendering for `api/*`, but the authentication middleware attempts to resolve a named login route before exception rendering. `routes/web.php` defines no named login route.

Impact

- Generic API clients, uptime monitors, scanners, and malformed requests inflate the 5xx error rate.
- Authentication failures are misclassified as server faults.
- Development responses disclose avoidable framework diagnostics.

Recommended fix

Configure guest redirects so API routes never resolve a web login route, or explicitly render `AuthenticationException` as 401 for every `api/*` request independent of content negotiation. Add regression tests using non-JSON `get()` requests for both tenant and vendor protected routes.

Acceptance criteria

- Every unauthenticated `/api/*` request returns 401 regardless of `Accept` header.
- No `Route [login] not defined` exception appears in API logs.

QA-003 - Authentication refresh effects produce lint warnings

Severity: Low **Priority:** P2 **Status:** Confirmed by lint

Oxlint reported react-hooks/exhaustive-deps warnings in:

- ProtectedLayout.tsx
- VendorProtectedLayout.tsx

Both effects use the currentUser object while declaring only currentUser.refetch as a dependency. The current implementation worked during browser testing, but the mismatch can create a stale event handler if the query result object changes.

Recommendation: destructure a stable refetch function and reference that inside the effect, or otherwise align the effect body and dependency list. Configure CI to fail on warnings if zero-warning lint is the project policy.

QA-004 - Critical UI flows are not protected by automated browser tests

Severity: Medium quality risk **Priority:** P2 **Status:** Coverage gap

The frontend suite contains only 8 tests covering the API helper, formatting utilities, and StatusBadge. There is no committed Playwright/Cypress suite for:

- Tenant or vendor authentication and logout
- First-login password flow
- Role-based navigation and forbidden states
- User/station/vendor onboarding forms
- Purchase order, delivery, sale, and reconciliation workflows
- CSV exports/report schedules
- Desktop/mobile routing regressions

Manual QA confirmed route-level behavior in this build, but CI cannot prevent regressions in those flows.

Recommendation: add a minimal browser suite covering one happy path and one authorization/error path for each principal workflow, starting with both login surfaces and the order-to-reconciliation journey.

5. Verified functional behavior

The following behavior was confirmed without observed defects:

- Tenant administrator login and live dashboard data
- Vendor Super User login and isolation from tenant navigation
- All 16 tenant workspaces rendered expected headings and controls
- All 3 vendor workspaces rendered expected headings and controls
- Vendor onboarding modal exposed labelled fields and blocked empty submission with native validation
- Desktop and mobile navigation behavior
- No browser console warnings/errors or Vite error overlays during the route smoke tests
- Existing backend coverage for login/logout/password reset, idle timeout, station and user management, tenant isolation, immutable audits, vendor licensing, shifts, email routing, scheduled reports, and the fuel order-to-reconciliation workflow
- CSRF rejection on login without a token (419)
- Correct unauthenticated response (401) when JSON is explicitly requested
- Allowed-origin credentialed CORS behavior and browser rejection of a nonmatching origin

- Redis connectivity (PONG) and recurring scheduler execution
- Local API health average of approximately 76 ms over 10 sequential requests
- Local web root average of approximately 9 ms over 10 sequential requests
- No tracked private-key/token patterns found by the targeted repository scan
- Production images use non-root runtime users and include the configured web security headers

6. Limitations and residual risk

This was a broad release-readiness assessment, not a formal penetration, accessibility, or load test.

- A real SMTP message was not sent because the active environment is connected to an external mail provider. Notification construction/routing passed automated tests, but delivery remains unverified until QA-001 is fixed and a dedicated test recipient is approved.
- The production Compose stack was validated and both images were built, but it was not launched against external production PostgreSQL/Redis/SMTP services.
- Browser QA avoided persistent CRUD changes; backend feature tests exercised the mutating operational workflows using SQLite.
- Backup/restore, disaster recovery, sustained concurrency, large data volumes, formal WCAG conformance, and external security testing were not executed.
- ATG hardware, offline sales synchronization, secure document storage, and queued PDF/XLSX delivery remain explicitly outside the current implementation baseline.

7. Recommended release sequence

1. Fix QA-001 and prove a full test run produces no live queue/mail activity.
2. Fix QA-002 and add content-negotiation-independent 401 regression tests.
3. Re-run all existing quality gates, production image builds, and the tenant/vendor browser smoke suite.
4. Clear or retain the 34 QA failed jobs only after the team confirms the desired evidence-retention policy.
5. Add the initial automated end-to-end suite before the next feature release.

Final verdict: the implemented product workflows are promising and mostly functional, but the build should not be released while automated tests can interact with live asynchronous infrastructure.